

Introducción a la Inteligencia Artificial
Proyecto sobre aprendizaje reforzado
Prof. Carlos B. Ogando M.

PROYECTO SMART TOP DOWN

HONESTIDAD ACADÉMICA

Como de costumbre, se aplica el código de honor estándar y la política de probidad académica. Las presentaciones isomórficas a (1) las que existen en cualquier lugar en línea, (2) las enviadas por sus compañeros de clase, o (3) las enviadas por los estudiantes en semestres anteriores serán consideradas plagio.

INSTRUCCIONES

En este proyecto desarrollarán agentes inteligentes que por aprendizaje reforzado pueden jugar contra un jugador en un juego de tipo Top Down.

I. Introducción

II. Diseño del agente

III. Qué necesita enviar

IV. Recomendaciones de abordaje

I. Introducción

1.1 Top Down

Un juego **top-down** (vista cenital o vista de pájaro) es un tipo de videojuego en el que la perspectiva de la cámara está ubicada directamente sobre la cabeza del personaje o la escena del juego, mirando hacia abajo. Este estilo de vista proporciona una visión completa del área de juego, permitiendo al jugador ver y controlar los movimientos del personaje en un plano bidimensional. Algunos elementos clave de los juegos top-down son:

1. **Perspectiva Vertical:** La cámara está posicionada directamente arriba, ofreciendo una vista clara de todo el entorno inmediato.
2. **Movimiento en 2D:** Aunque los juegos pueden tener gráficos en 3D, el movimiento del personaje suele estar limitado a un plano bidimensional (hacia arriba, abajo, izquierda o derecha).
3. **Visibilidad del Entorno:** Este estilo permite una amplia visibilidad del área circundante, facilitando la planificación y la estrategia.

Ejemplos de Juegos Top-Down

- **The Legend of Zelda:** Los primeros juegos de esta serie utilizaban una perspectiva top-down.
- **Grand Theft Auto:** Los primeros títulos de la serie (GTA y GTA 2) empleaban una vista cenital.
- **Hotline Miami:** Un juego de acción con una perspectiva top-down.

Aplicaciones Comunes

- **Juegos de Estrategia:** La vista top-down es común en juegos de estrategia en tiempo real (RTS) porque facilita la gestión de unidades y recursos.
- **Juegos de Rol (RPG):** Muchos RPG utilizan esta perspectiva para permitir una fácil navegación y exploración del mundo del juego.
- **Shooters y Juegos de Acción:** Algunos juegos de acción y disparos también utilizan esta vista para ofrecer un campo de visión amplio y dinámico.

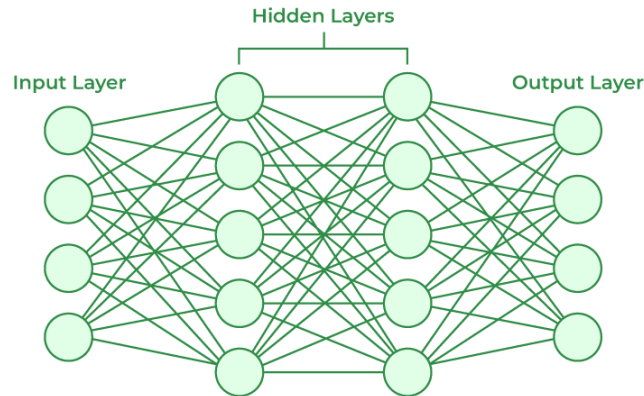
Plantear el agente inteligente desde sus sensores y actuadores es fundamental para poder implementar el mismo. Los actuadores representan las acciones del juego y los sensores las observaciones del entorno.

II. Diseño del agente

El objetivo de este proyecto es crear enemigos inteligentes que puedan jugar contra el jugador en un juego estilo Top Down. Para lograr esto, utilizaremos una combinación de técnicas avanzadas de inteligencia artificial: aprendizaje por refuerzo (RL) y redes neuronales optimizadas mediante algoritmos genéticos.

2.1. Componentes del diseño

1. **Sensores del juego (entradas de la red neuronal):** el agente debe ser capaz de percibir su entorno en el juego para tomar decisiones informadas. Para ello, se utilizarán varios sensores que indicarán el estado actual del jugador. Algunos ejemplos de qué puede medirse son:
 - **Distancia al jugador:** La distancia del agente al jugador.
 - **Salud:** Salud del agente.
 - **Estado del escudo:** Salud del escudo del agente.
2. **Actuadores (salidas de la red neuronal):** el agente actuará en su entorno a través de diversas acciones dependiendo del tipo de enemigo que sea. Algunas de estas pueden ser:
 - **Escapar:** Escapa del jugador.
 - **Perseguir:** Persigue al jugador.
 - **Atacar:** Ataca al jugador
 - **Lanzar:** Lanza un objeto hacia el jugador.
3. **Red Neuronal (cerebro del agente)** La red neuronal será la encargada de procesar la información recibida de los sensores y determinar la acción a tomar. La estructura de la red neuronal incluirá:
 - **Capa de entrada:** Una neurona para cada sensor del juego.
 - **Capas ocultas:** varias capas intermedias que procesarán la información de manera no lineal.
 - **Capa de salida:** Una neurona para cada acción del juego.



4. **Aprendizaje por refuerzo (RL):** El agente aprenderá a jugar contra el jugador mediante el aprendizaje por refuerzo implícito en los algoritmos genéticos. Este enfoque se basa en la idea de que el agente recibirá una recompensa o penalización según su desempeño en el juego (ser seleccionado o no). El objetivo del agente es maximizar la recompensa acumulada a lo largo del tiempo. Las principales características del aprendizaje por refuerzo son:
 - **Estado:** La información recibida de los sensores en un momento dado.
 - **Acción:** La decisión de que acción realizar (escapar, atacar, defenderse, entre otras)
 - **Recompensa:** Una puntuación que refleja el desempeño del agente (por ejemplo, +1 por cada vez que ataca el jugador, +1 cada vez que esquiva, -1 cada vez que recibe daño)
5. **Optimización con algoritmos genéticos** Para mejorar el desempeño de la red neuronal, utilizaremos algoritmos genéticos, los cuales imitan el proceso de evolución natural. Este método permite optimizar los pesos de la red neuronal a través de:
 - **Selección:** Los mejores agentes (redes neuronales con mayor desempeño) son seleccionados para reproducirse.
 - **Cruce:** Combinación de las características de dos agentes seleccionados para crear nuevos agentes.
 - **Mutación:** Introducción de pequeñas modificaciones aleatorias en algunos agentes para mantener la diversidad genética y explorar nuevas soluciones.

2.2. Proceso de entrenamiento

1. **Inicialización:** Crear una población inicial de agentes con redes neuronales con pesos aleatorios.

2. **Simulación:** Permitir que cada agente juegue múltiples partidas del juego, registrando su desempeño.
3. **Evaluación:** Calcular la recompensa acumulada de cada agente y seleccionar los mejores.
4. **Reproducción:** Aplicar los algoritmos genéticos (selección, cruce y mutación) para crear una nueva generación de agentes.
5. **Repetición:** Repetir el proceso de simulación, evaluación y reproducción durante muchas generaciones hasta que el agente muestre un desempeño óptimo.
6. **Validación:** Validar el desempeño obtenido en niveles nunca vistos por el agente.

III. Qué necesita enviar

En este proyecto crearán varios agentes inteligentes usando máquinas de estado finito para modelar varios enemigos en un juego Top Down estilo Hotline Miami. Implementarán algoritmos de pathfinding.

Los agentes deben aprender a jugar como enemigo del jugador empleando aprendizaje reforzado a través de una red neuronal optimizada por algoritmos genéticos.

3.1. Requerimientos de juego e IA:

1. El juego será de tipo Top Down, estilo Hotline Miami.
2. El jugador debe poder realizar las siguientes acciones:
 - a. Moverse en las cuatro direcciones cardinales.
 - b. Atacar de cerca (meele attack).
 - c. Lanzar un proyectil (ranged attack).
 - d. Sanarse con una pócima que encuentre.
 - e. Defenderse (esta acción hace que el jugador no pueda moverse).
3. Deberá crear los siguientes enemigos:
 - a. Tipo A. Un enemigo con la capacidad de que cuando detecte a un jugador en su campo de visión, entonces puede perseguirlo y atacarlo melee hasta tratar de matarlo. Si su salud es muy baja, trata de buscar una pócima en el nivel para sanarse.
 - b. Tipo B. Un enemigo que cuando detecta al jugador en su campo de visión no se mueve, sino que lo ataca a distancia y cuando el jugador se acerca, se defiende.
 - c. Tipo C. Un enemigo que cuando detecta a un jugador en su campo de visión lo ataca a distancia, si el jugador se acerca, se escapa del jugador, y si el jugador está demasiado cerca lo ataca de vuelta. Si su salud es muy baja, trata de buscar una pócima en el nivel para sanarse.
 - d. El enemigo entrenado por RL que tenga todas las acciones disponibles.
4. Los enemigos deben tener implementados algoritmos de pathfinding que permitan moverse adecuadamente en el nivel.
5. Debe haber un obstáculo tipo "púa" que "mate" al jugador o a los enemigos cuando colisionen con el mismo.
6. Deben existir bloques colisionables de tipo pared.
7. Deben existir pócimas de sanación en el nivel que pueda ser usada por cualquiera.
8. La acción de defensa del jugador gasta su escudo hasta romperse cuando se ha usado mucho.
9. Crearán 3 niveles base en donde se pueda ver el funcionamiento de todo lo requerido.
10. El juego debe verse gráfica y sonoramente adecuado.

3.2. Requerimientos del optimizador:

- Debe existir tantos actuadores como acciones posibles tiene el enemigo.
- Deben existir al menos cinco sensores.
- La red neuronal de cada agente debe tener 1 capa de entrada con los sensores creados, 1 capa de salida con los outputs deseados, y mínimo 1 hidden layer. (3 capas mínimo en total)
- La optimización debe ser usando algoritmos genéticos. Las funciones de cruce y mutación se deciden de forma arbitraria.
- La función de rendimiento debe ser en base la recompensa del agente. La recompensa del agente será definida de forma arbitraria. Algunos ejemplos de factores que pueden influir en la recompensa del agente son: tiempo de vida, salud, ataques al jugador, daños provenientes del jugador, uso de pócimas, etc.
- Debe probar al menos con 1 función de cruce.
- Debe probar al menos con 2 funciones de mutación.
- Debe probar al menos con 2 funciones de selección.

3.3. Requerimientos de la validación:

1. Para el entendimiento y el análisis de su desarrollo entrenarán al enemigo de forma reforzada usando a los otros enemigos y al jugador humano.
Específicamente se debe entrenar el agente con:
 - a. Enemigo tipo A.
 - b. Enemigo tipo B.
 - c. Enemigo tipo C.
 - d. Jugador humano.
2. Deberán validar con diferentes combinaciones y cantidades de oponentes para hacer pruebas de estrés al agente entrenado.
3. Realizarán un benchmark cualitativo del agente inteligente en su proceso de experimentación, describiendo el rendimiento de este con los diferentes enemigos y con el jugador humano.
4. Realizarán un benchmark cuantitativo en donde compararán:
 - a. Escenarios:
 - i. Un enemigo tipo A vs 1 agente.
 - ii. Un enemigo tipo B vs 1 agente.
 - iii. Un enemigo tipo C vs 1 agente.
 - iv. Un jugador humano vs 1 agente.
 - v. Varios enemigos tipo A vs 1 agente.
 - vi. Varios enemigos tipo B vs 1 agente.
 - vii. Varios enemigos tipo C vs 1 agente.
 - viii. Varios de todos los enemigos vs 1 agente.
 - ix. Varios de todos los enemigos vs varios agentes.
 - x. Un jugador humano vs varios agentes.
 - b. Métricas:
 - i. Ganador del conflicto.

- ii. Tiempo de vida.
- iii. Daño por segundo (eficiencia del agente)
- iv. Tasa de éxito (ratio de cuántos mató antes de que lo mataran del total)
- c. Variables para comparar:
 - i. Número de pobladores.
 - ii. Tasa de mutación (probabilidad).
 - iii. Configuración de peso inicial. (aleatorio, pesos iguales, pesos nulos...)
 - iv. Tasa de selección.
 - v. Técnicas de mutación. (con sus parámetros respectivos) (2 o más)
 - vi. Técnicas de selección. (con sus parámetros respectivos) (1 o más)
 - vii. Técnicas de cruce. (con sus parámetros respectivos) (1 o más)
 - viii. Técnica de thresholding de la red neuronal para activar las acciones. (1 o más)
- d. Ejemplo de cómo se puede ver:

Un enemigo tipo A vs 1 agente	Ganador del conflicto	Tiempo de vida	Daño por segundo	...	Tasa de éxito
Número de pobladores = 10					
Número de pobladores = 20					
...					
Prob de mutación = 0.1					
Prob de mutación = 0.2					

Nota: No limitarse a este esquema. Pueden hacer una tabla nivel vs variable bajo una sola métrica ordenado por dificultad.

Tasa de éxito	Un enemigo tipo A vs 1 agente	Un enemigo tipo B vs 1 agente	Un enemigo tipo C vs 1 agente	...	Un jugador humano vs varios agentes
Número de pobladores = 10					
Número de pobladores = 20					
...					
Prob de mutación = 0.1					

Prob de mutación = 0.2					
------------------------------	--	--	--	--	--

Pueden acompañar el análisis de los algoritmos con gráficas o cualquier otro elemento que consideren importante.

IV. Recomendaciones de abordaje

1. Lo ideal es que el proceso de entrenamiento del agente se realice de forma incremental escenario en escenario. Al iniciar el entrenamiento de un escenario tomar el mejor agente del anterior.
2. Para lograr convergencia más rápido, frisar la optimización de los sensores relacionados a mecánicas no relacionadas con el nivel es muy recomendable.
3. También probar con una red neuronal shallow (solo capa de entrada y de salida) es sumamente recomendable.
4. Importante recordar normalizar las salidas de las neuronas para que las ponderaciones tengan sentido.